



UNIVERSITY OF
ILLINOIS
URBANA-CHAMPAIGN

SE 423: Introduction to Mechatronics

Lecture 12: IMU

Marius Juston

Monday, February 2nd, 2026

Talk to the person next to you and answer these review questions.

- To receive data, you must also _____ data.
- What is FIFO? Why is it used?
- What is the maximum size for the buffer's data in bits?
- What does SPICHR = 0xF mean?
- What do absolute / incremental encoders measure? When do they fail at measuring travel distance?

Office Hours:

- **Monday:** 4-6 PM, Neel
- **Tuesday:** 11-1 PM, Lakshmi
- **Tuesday:** 1-3 PM, Samuel
- **Tuesday:** 2-5 PM, Dan & Marius

Lab: Starting Lab 4 this week. This a 2-week lab!

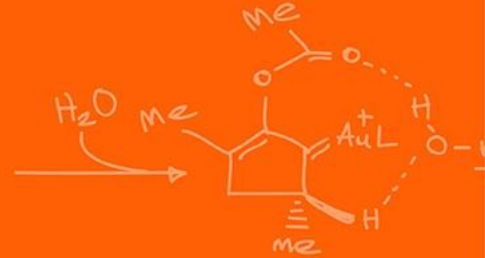
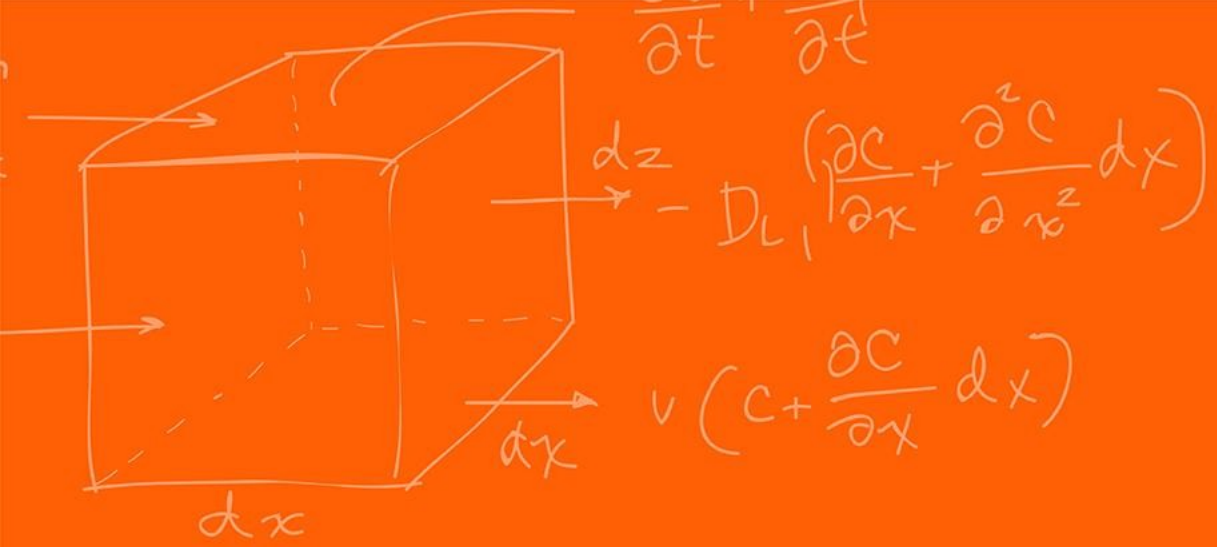
Homeworks:

- Check-offs for [homework 3](#) Ex 1, 2, 3 & 4 are due **March 10, 5PM (The end of office hours)**
- Check-offs for [homework 3](#) Ex 5 are due **March 24, 5PM (The end of office hours)**
- Answers and code submissions for [homework 3](#) are due **March 25, 9AM**

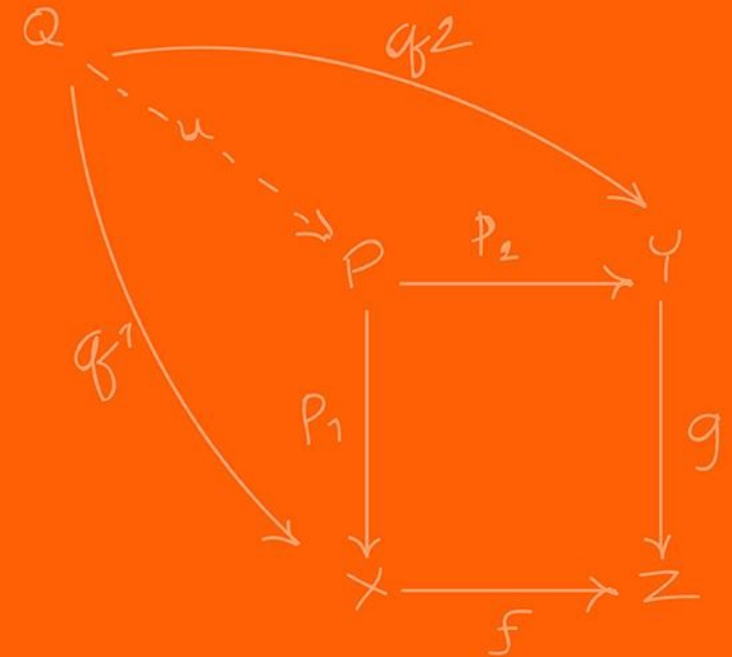
Questions?

Homework, Lab, LabVIEW?

**Thank you to everyone that
responded to my feedback form!
There was a lot of great feedback
that I will try to incorporate!**



Accelerometer



We cannot and should not rely on a single sensor source (sensor fusion will be covered later), and sometimes we cannot attach encoders.

So, what can we do?

The goal is to measure direction and rotation. What is the most basic component that we can measure?

Forces!

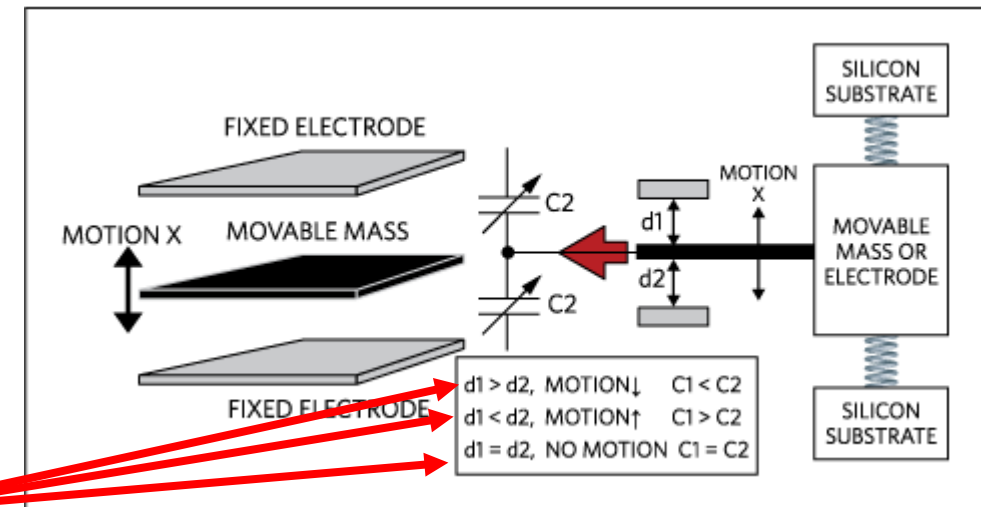
How do calculate velocity and position from forces?

A MEMS (Micro-Electro-Mechanical Systems) accelerometer measure linear acceleration.

Accelerometers work like a weight suspended by springs. When the mass is moved you measure the displacement, the springs bring it back to the center.

In chips, we use the change of capacitors. The capacitance is based on the distance between the 2 materials. So, the smaller the distance, the larger the returned capacitance.

You then measure this capacitance.



<https://www.analog.com/cn/resources/technical-articles/accelerometer-and-gyroscopes-sensors-operation-sensing-and-applications.html>

Force causes a displacement of the mass which, in turn, causes a capacitance change.

Placing multiple electrodes in parallel allows for a larger capacitance, which will be more easily detected.

By calculating the difference between C_1 and C_2 , we can derive the displacement of the mass and its direction.

Since we can measure the displacement and the system acts as a string we can do the following. We start with:

$$F = ma$$

The force from a spring is:

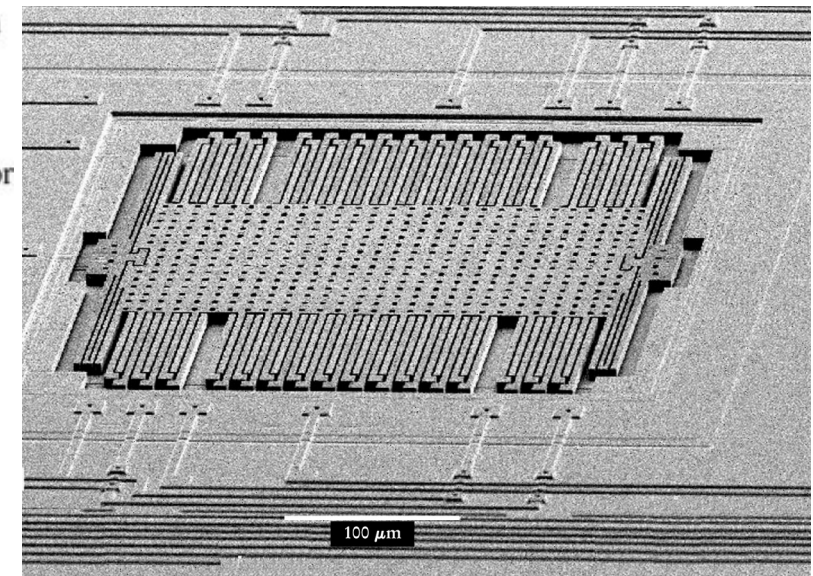
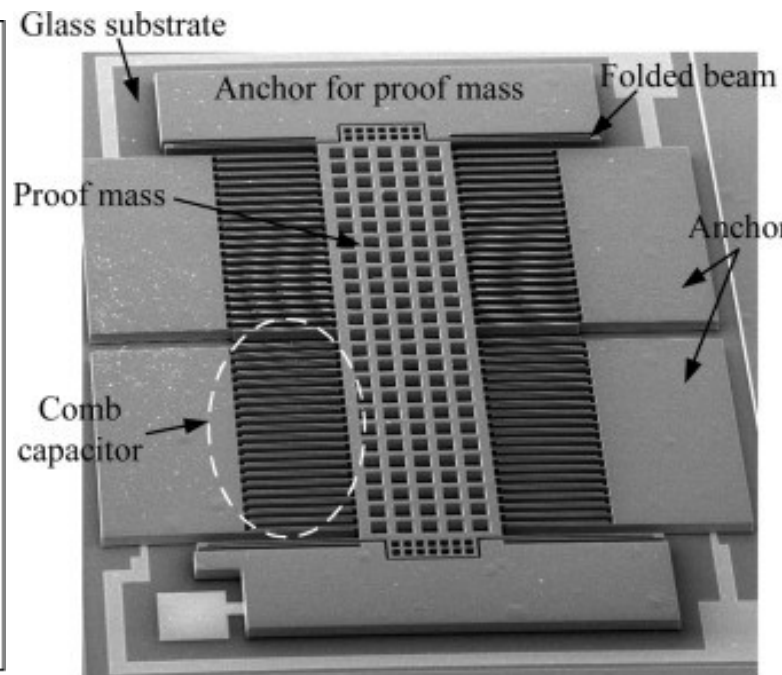
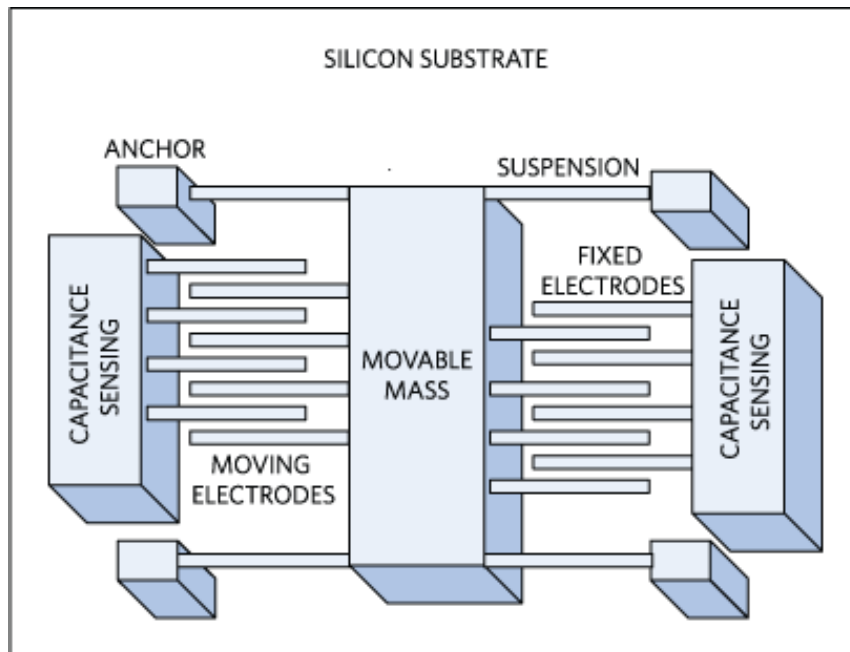
$$F = k \Delta x$$

Given that m, k are known (design choices), and Δx is measured from the capacitors, we get that:

$$F = ma = k\Delta x$$
$$a = \frac{k}{m} \Delta x$$

You keep stacking these to remove the noise to get more accurate readings. In turn we get the following internal system. It is incredible how small these are!

How many axis can the following accelerometers measure?



<https://www.analog.com/cn/resources/technical-articles/accelerometer-and-gyroscopes-sensors-operation-sensing-and-applications.html>

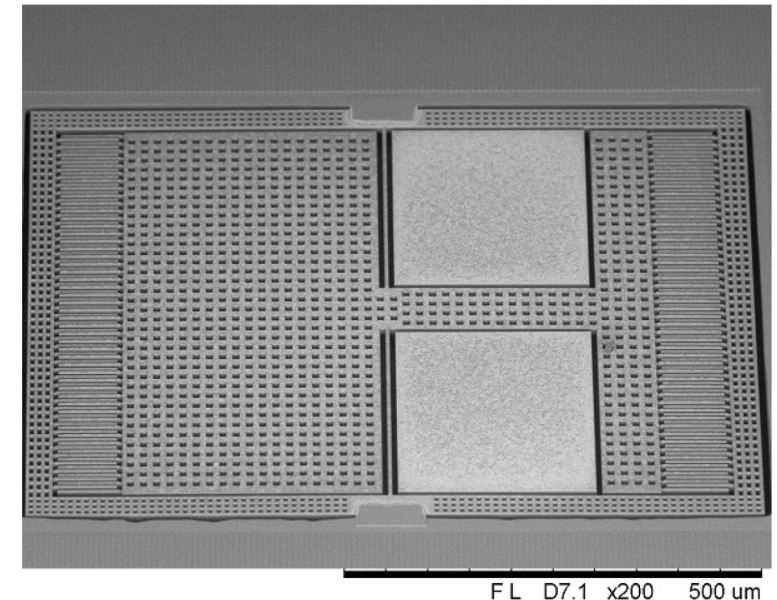
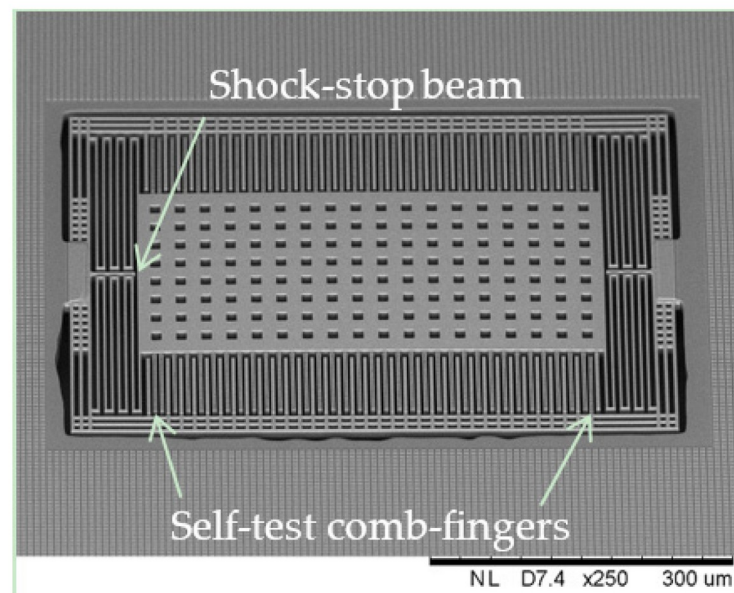
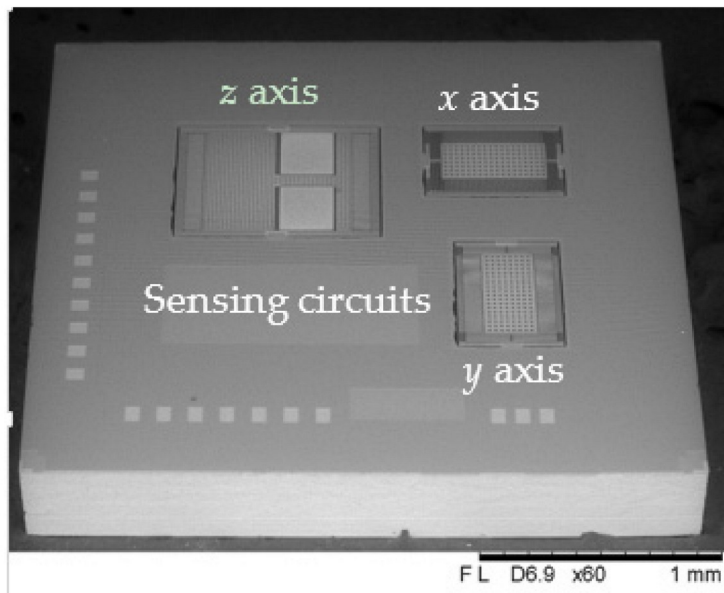
<https://www.sciencedirect.com/science/article/pii/S0924424716300267>

<https://physicstoday.aip.org/features/the-little-machines-that-are-making-it-big>

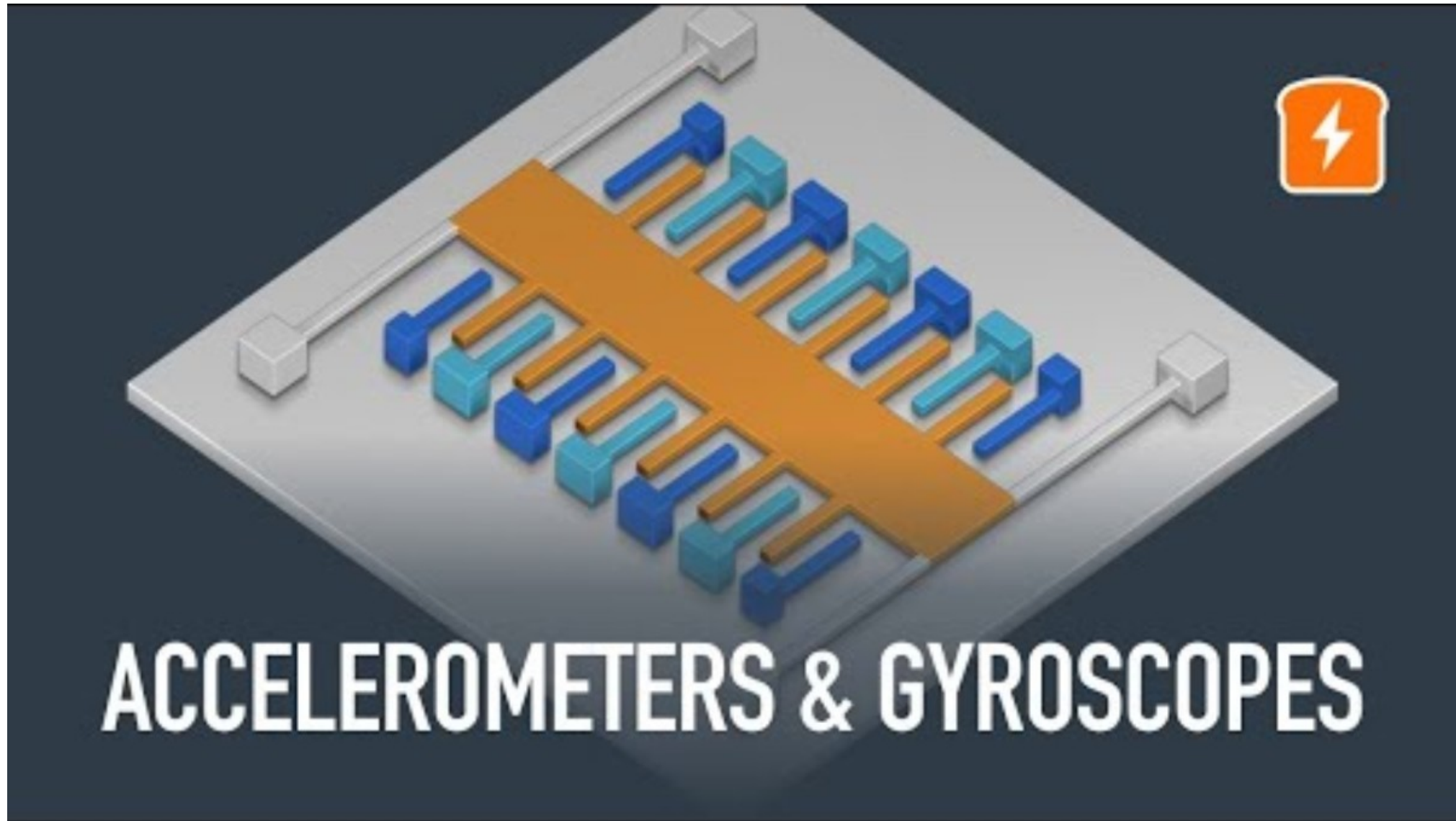
Think of the z-axis as a trampoline!

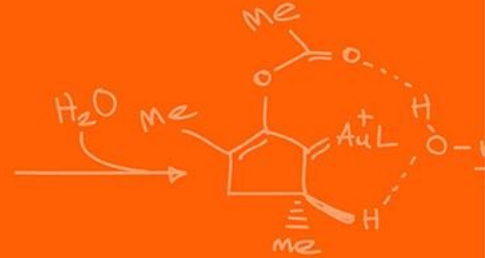
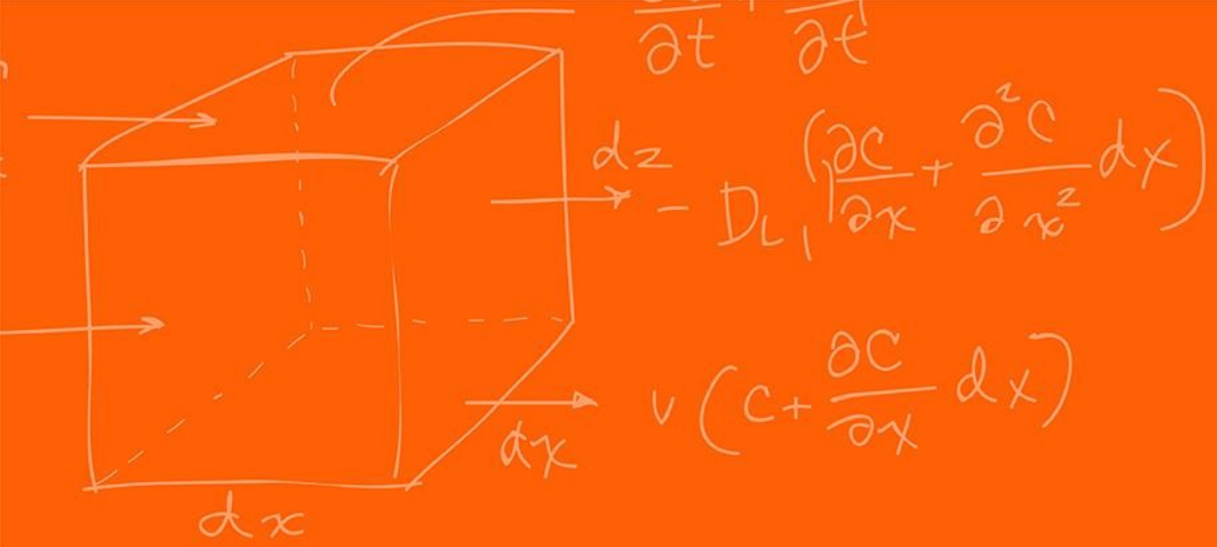
What happens if you have too large of an acceleration (think of the combs)?

An accelerometer that is sitting still on a desk reads $1g$ pointing upwards.
What if it gets dropped out of the window in a perfect vacuum?

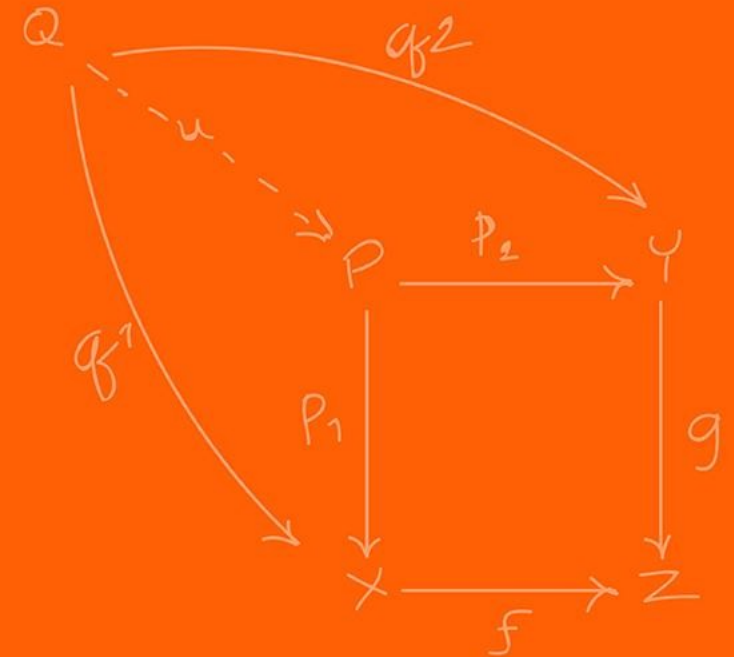


<https://www.mdpi.com/2504-3900/1/4/337>





Gyroscope



The accelerometer measures linear acceleration, which allows us to measure acceleration, velocity and displacement.

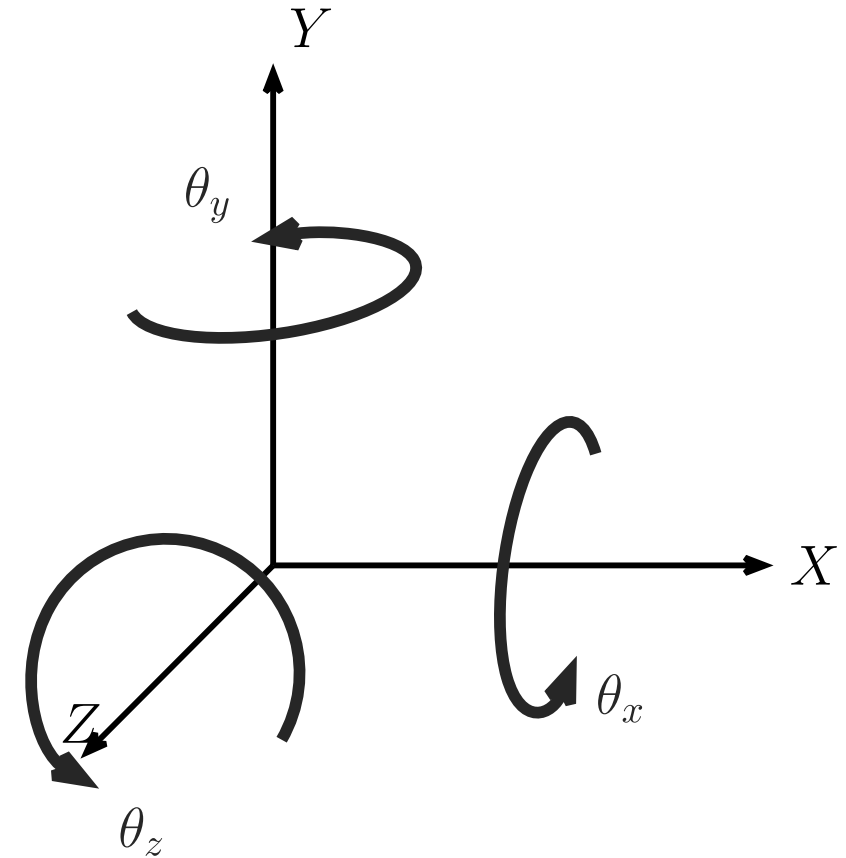
How many degrees of freedom are there in 3D?

$$(x, y, z, \theta_x, \theta_y, \theta_z)$$

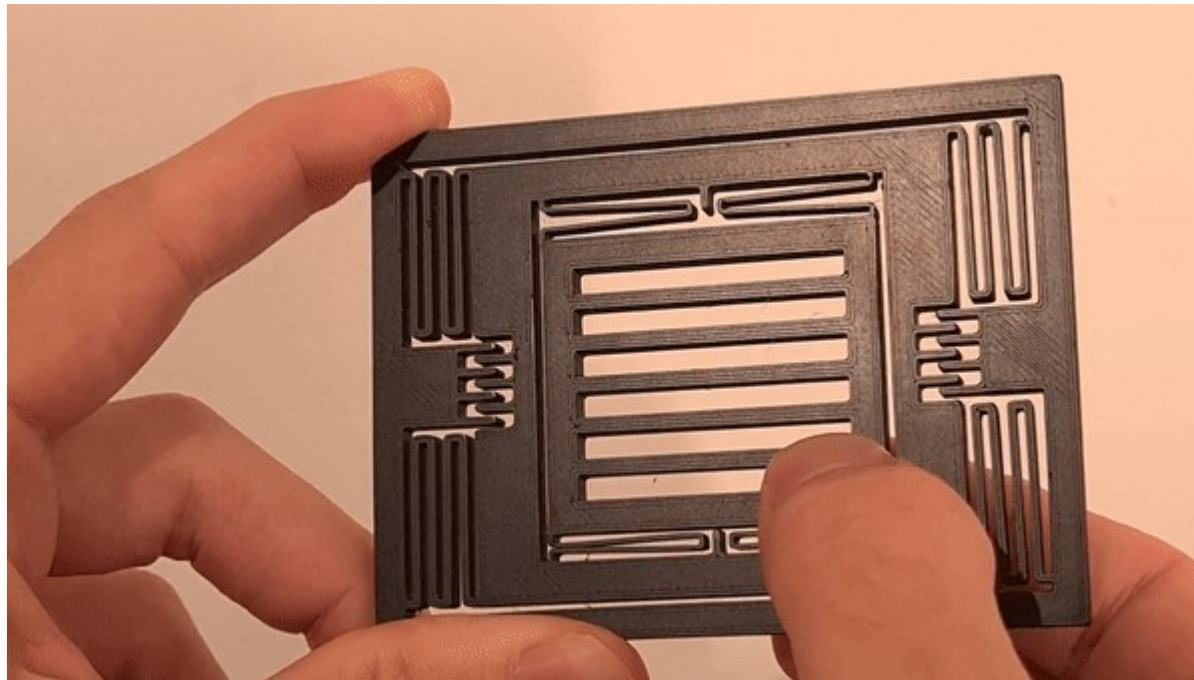
How many does the accelerometer handle?

$$3: (x, y, z)$$

We are missing the angular DOF!



A MEMS Gyroscope does not use a spinning wheel like classical mechanical gyroscopes. Similarly, you have a tiny mass that is moving when rotating. If the device start rotating, the moving mass experiences a sideways force.



https://www.reddit.com/r/3Dprinting/comments/103v6ef/1001_3d_printed_mems_gyroscope/

Very similar to accelerometers,

The mass sits between capacitors plates.

When the it moves sideways, capacitance changes.

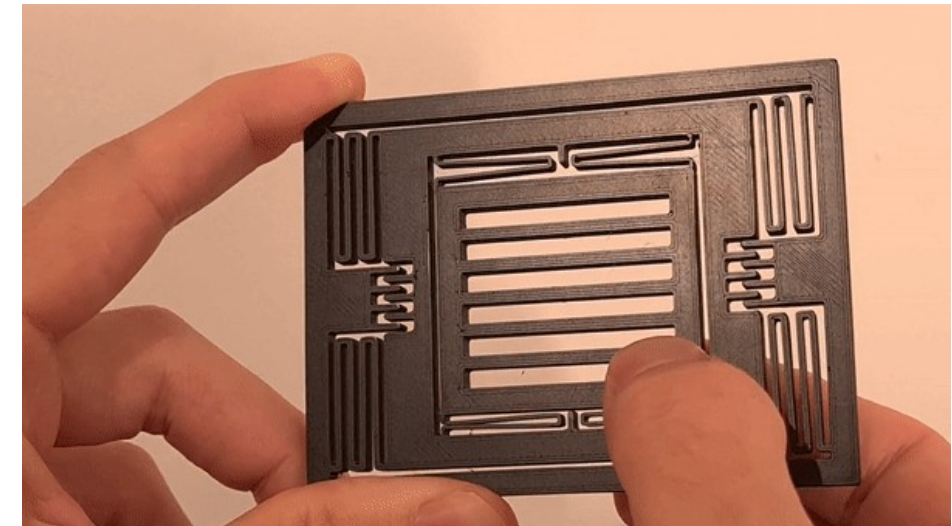
This returns a displacement from which we can compute angular velocity.

The device shown has 2 axis, the **Drive Axis** (x) and the **Sense Axis** (y).

The inner mass is electrically forced to vibrate continuously back and forth along the x-axis at a resonant frequency ω_r . The instantaneous velocity is v .

When an angular velocity Ω_z , along the z-axis, is applied, the Coriolis force acting along the y-axis is generated.

$$F_C = -2m(\Omega_z \times v)$$



https://www.reddit.com/r/3Dprinting/comments/103v6ef/1001_3d_printed_mems_gyroscope/

-

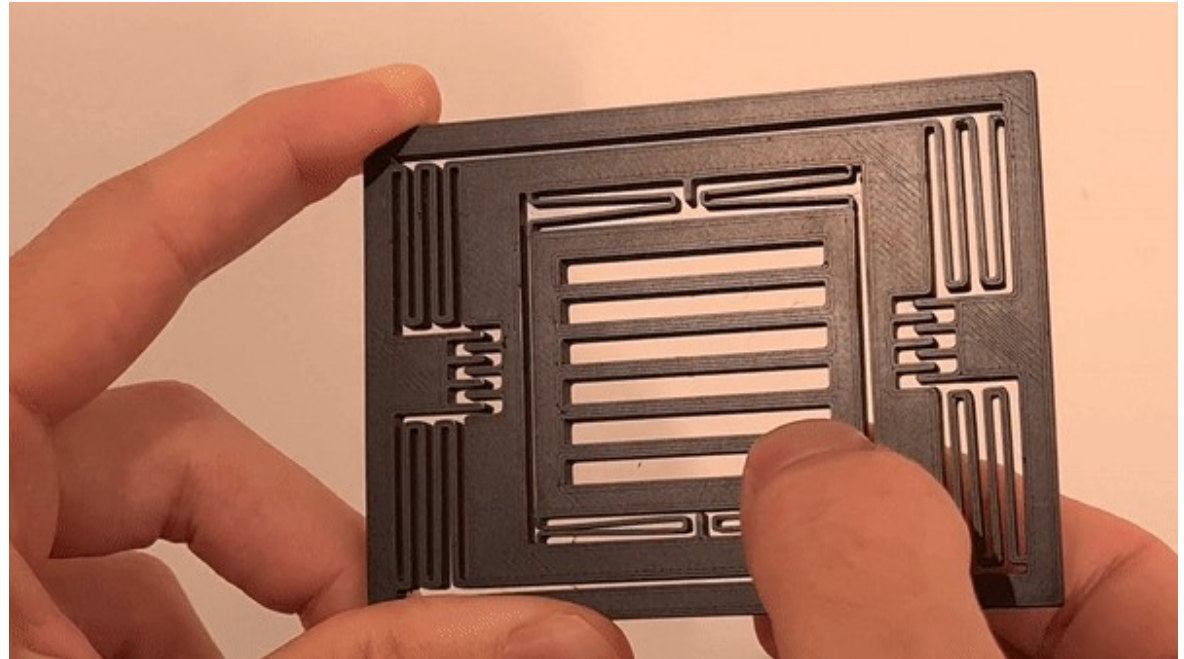
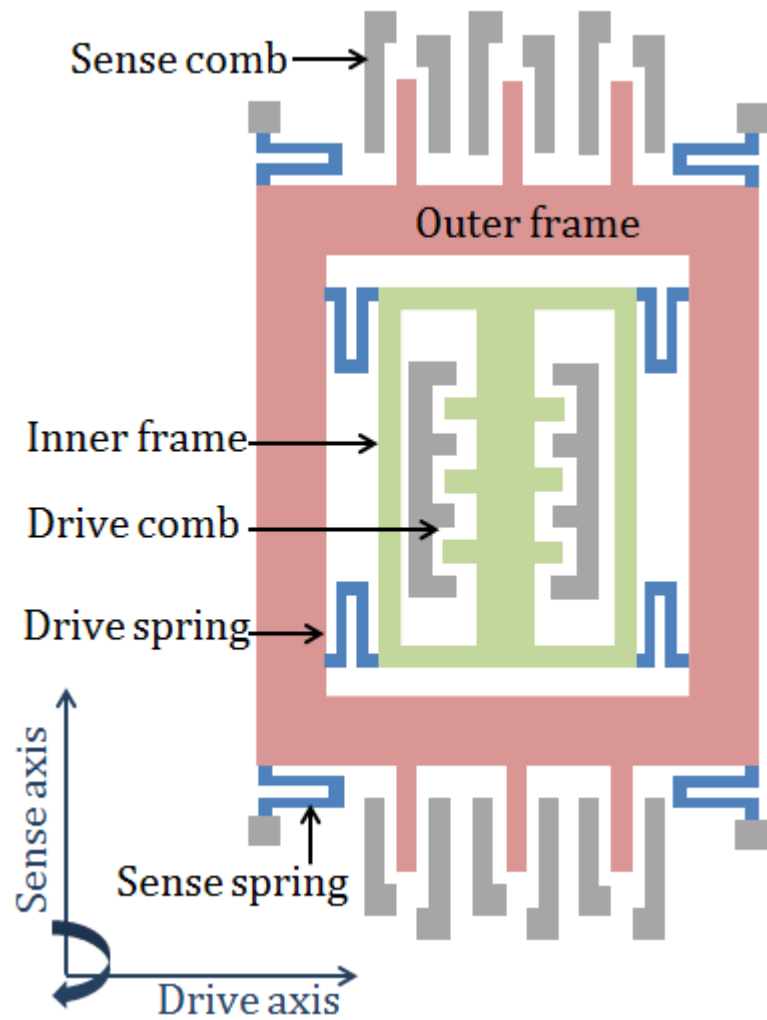
The Coriolis force pushes the mass along the y-axis.

Thus, to solve for angular velocity we again derive that:

$$F = k\Delta y = -2m(\Omega_z \times v)$$
$$\Omega_z = \frac{k\Delta y}{2m v}$$

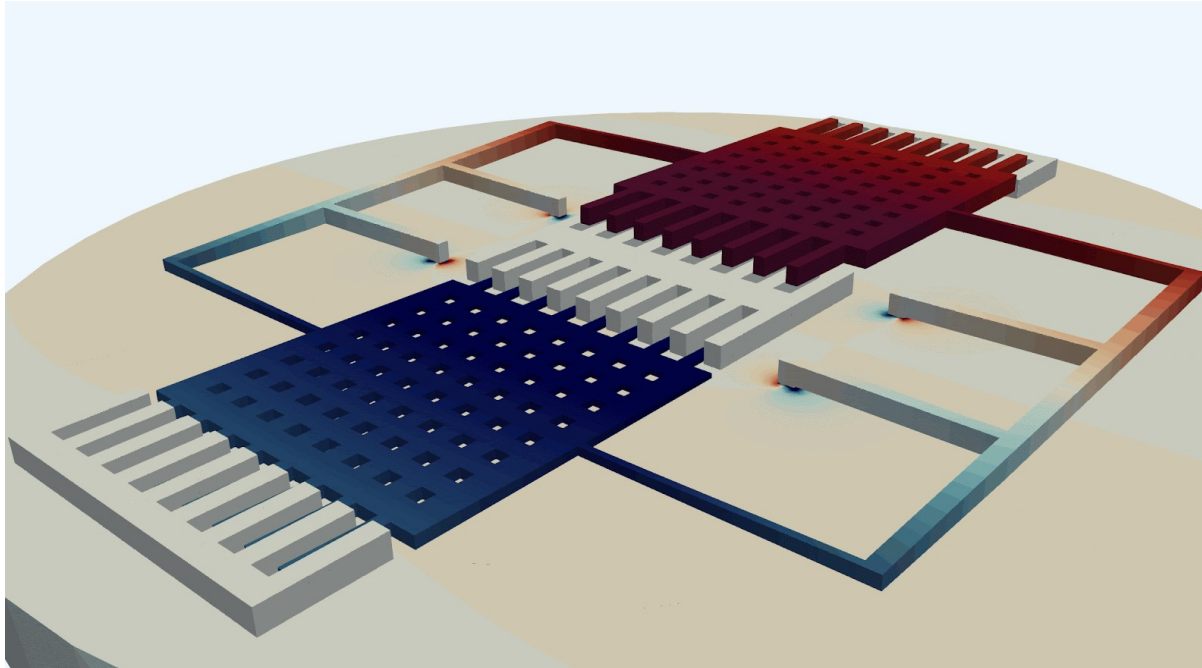
We know the mass m , spring stiffness k , drive velocity v , so we just compute the displacement Δy to measure Ω_z .

Very similar to the accelerometer!

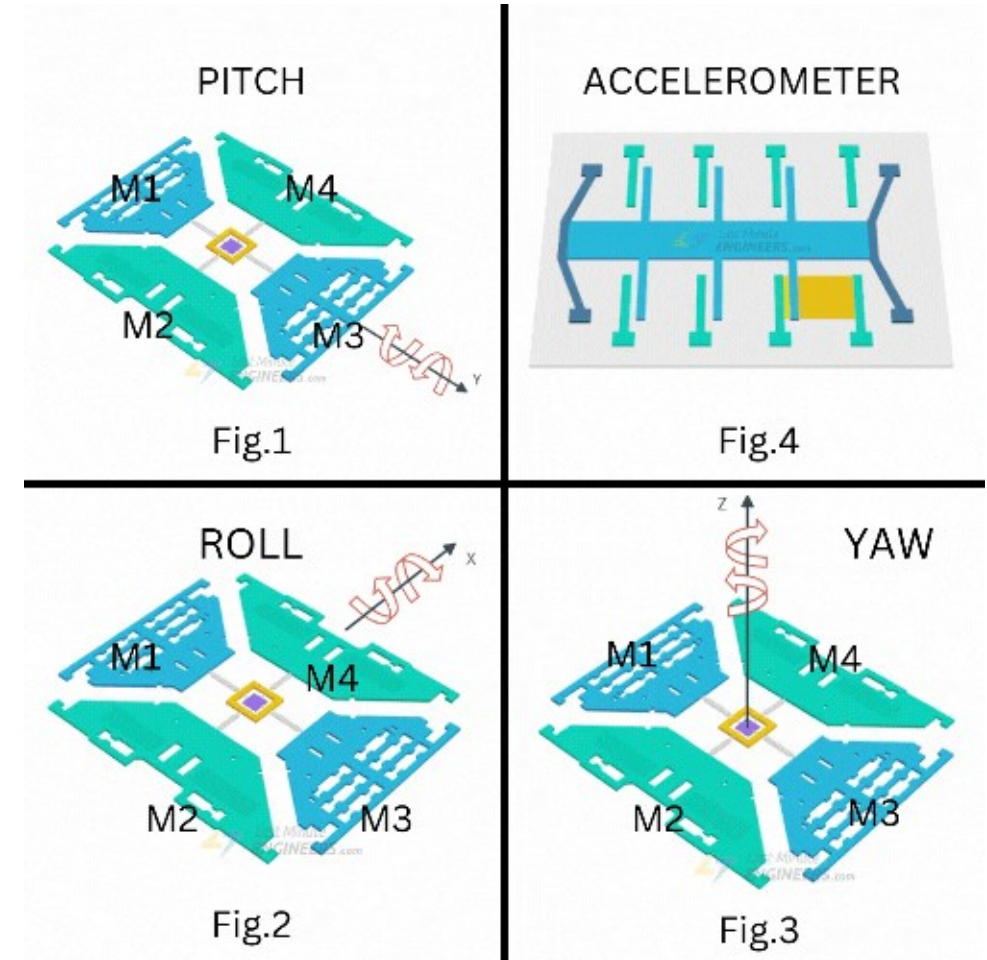


<https://www.semanticscholar.org/paper/Modeling-and-simulation-of-the-MEMS-vibratory-Patel-McCluskey/8dec6605fc5b603df07d74f7b9eb6875b2c17c3a>

https://www.reddit.com/r/3Dprinting/comments/103v6ef/1001_3d_printed_mems_gyroscope/

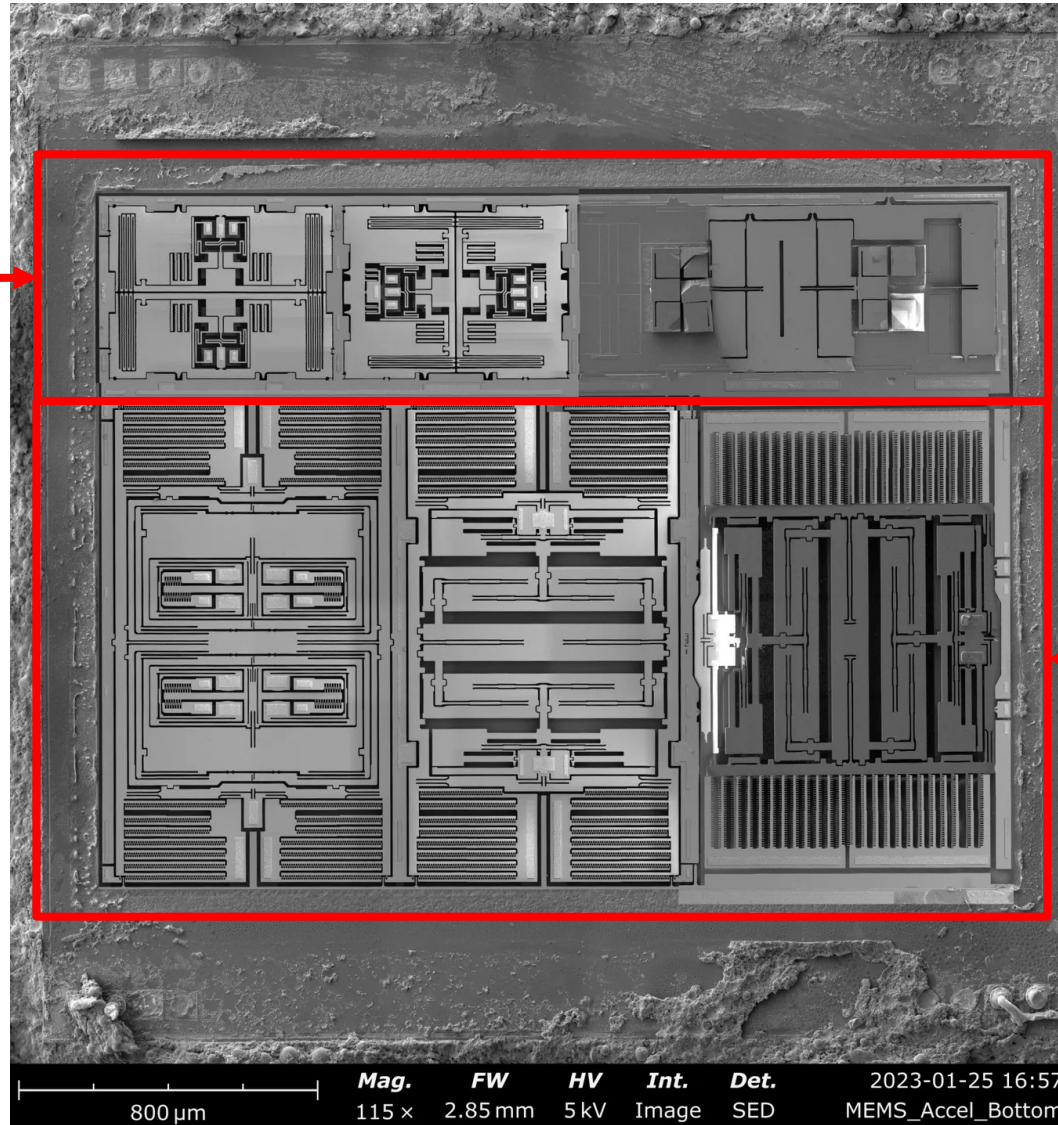


<https://quanscient.com/blog/enhancing-mems-gyroscope-design>



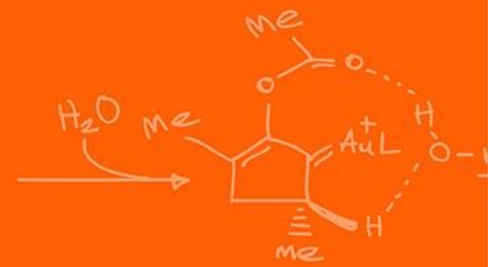
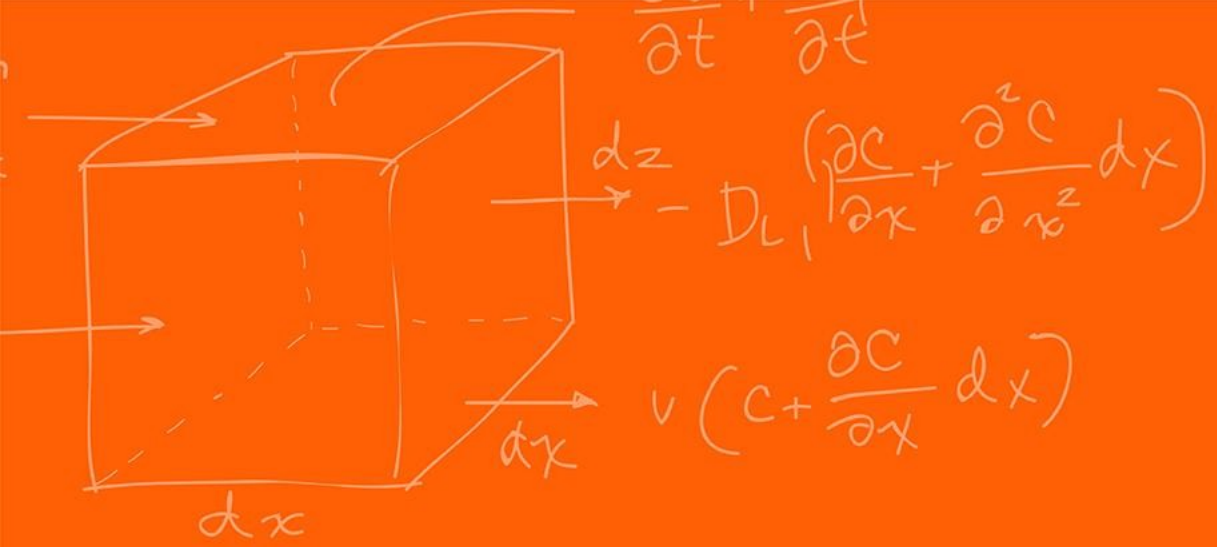
<https://forum.arduino.cc/t/getting-started-with-the-mpu6050-beginner-friendly-guide-with-visual-explanations/1364064>

What is this? The gyros or accelerometers?

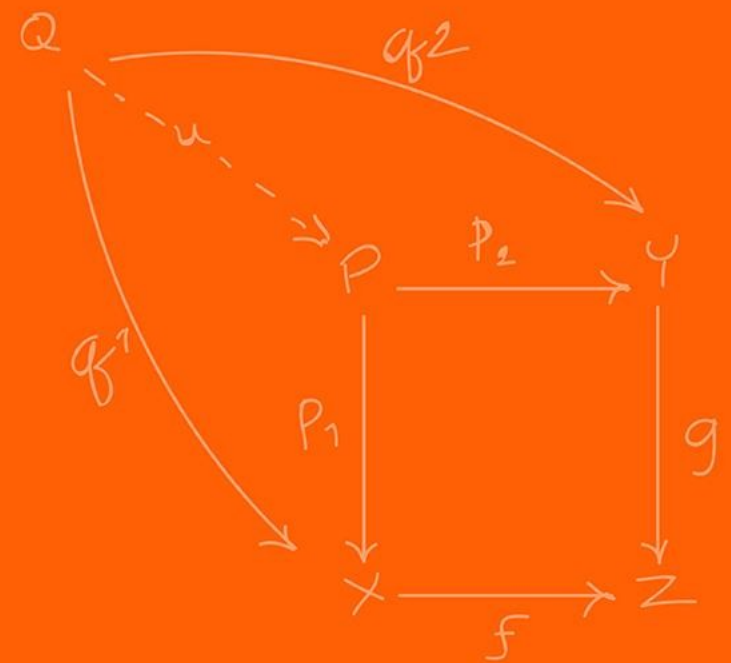


What is this? The gyros or accelerometers?





Magnetometer



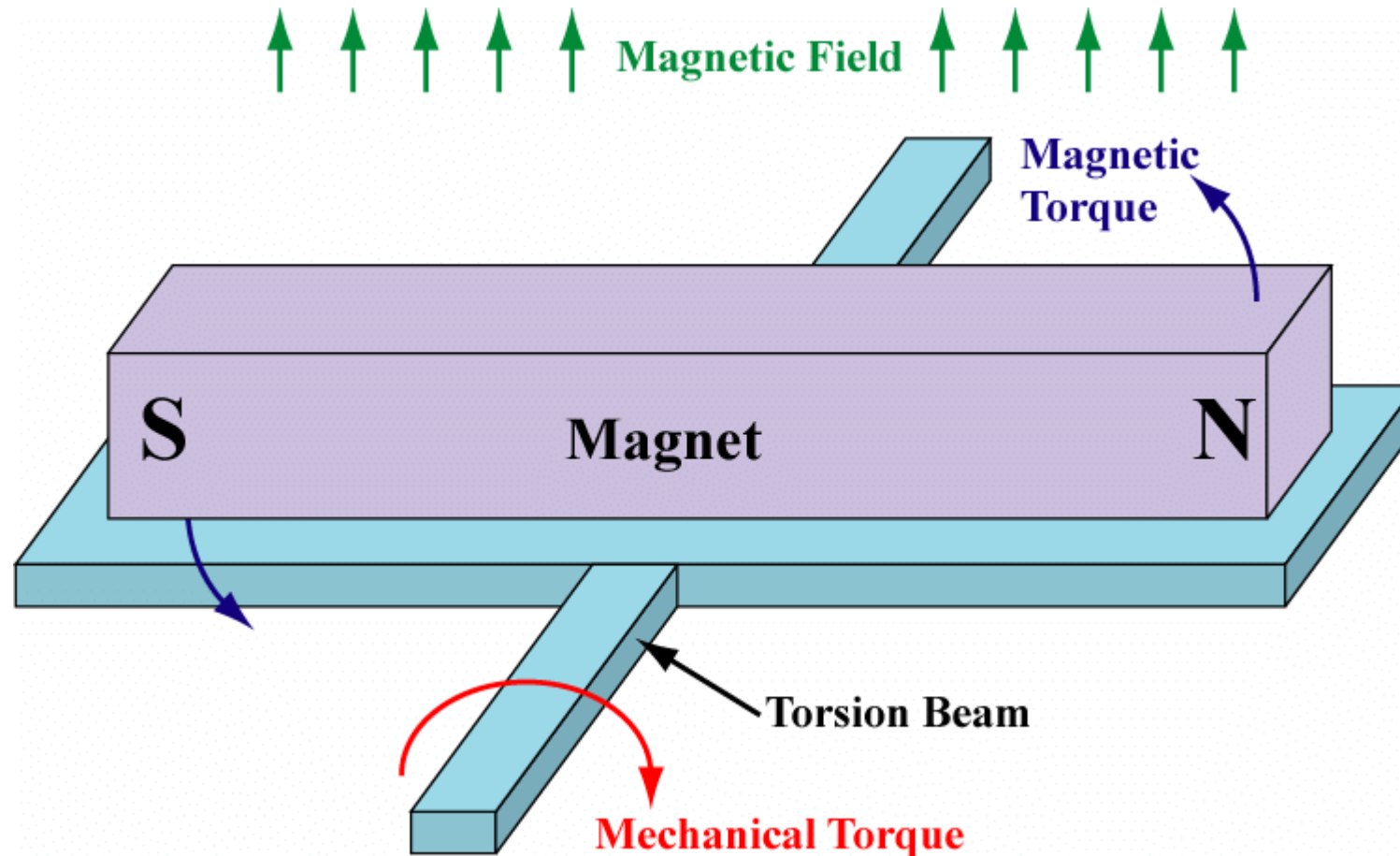
What is a device that has been used for traveling for a very long time? That has helped sailors travel?

The compass!

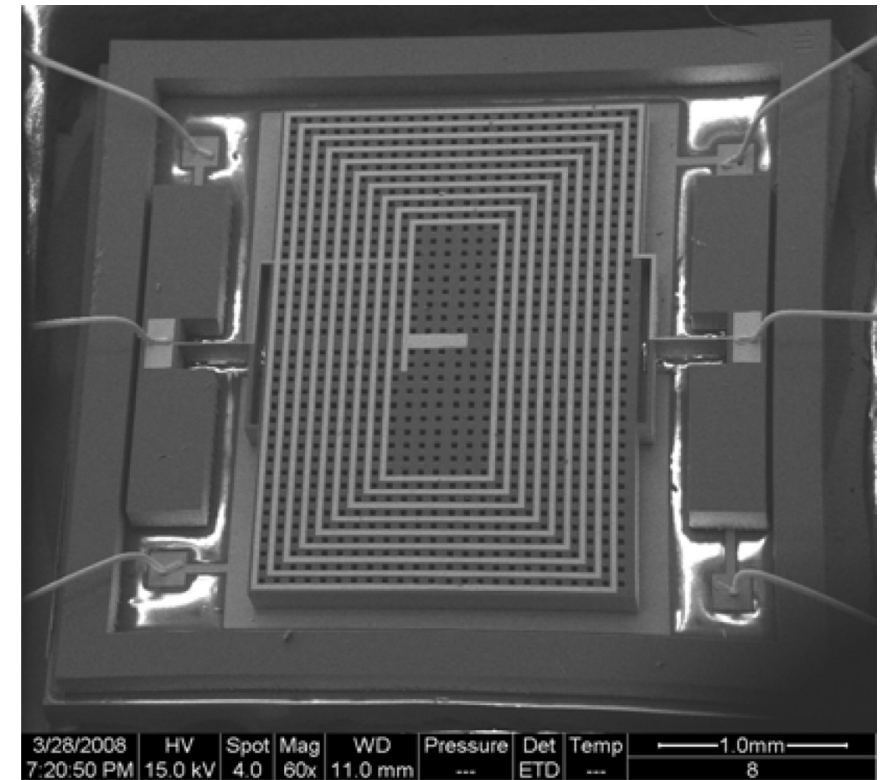
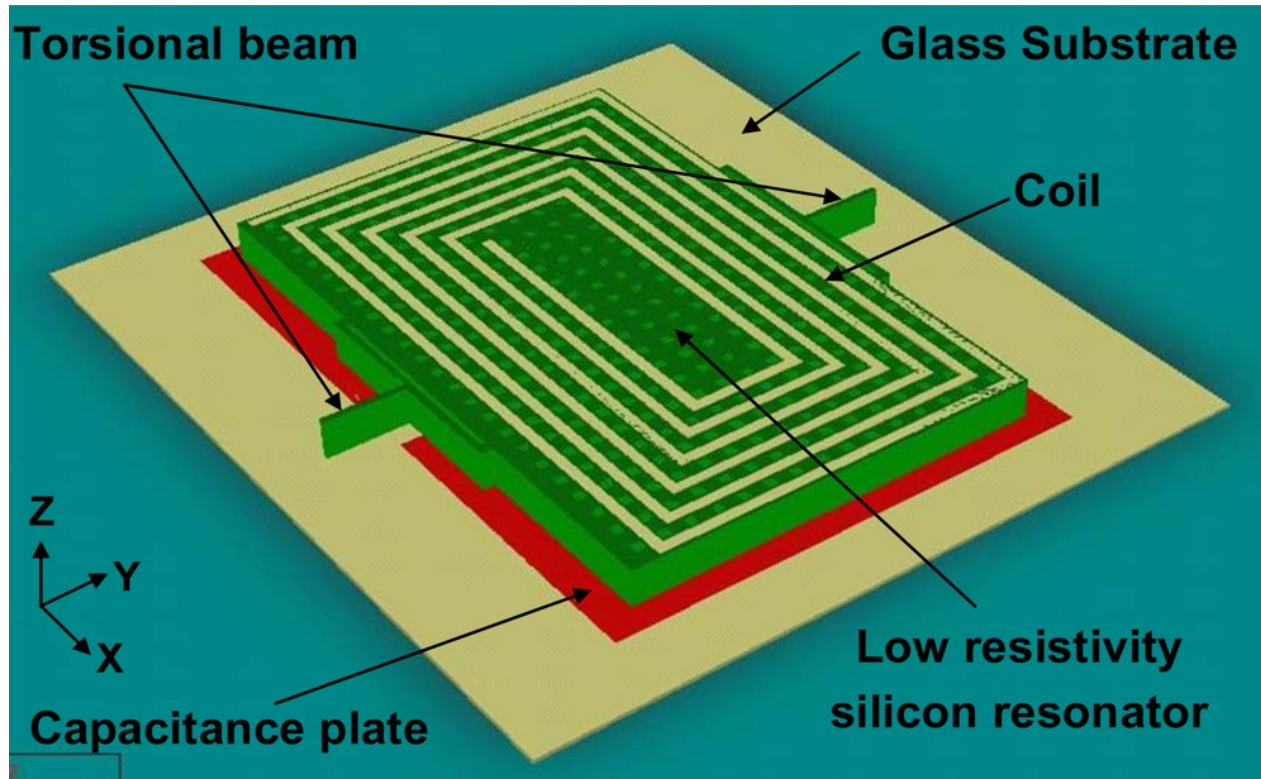
The magnetometer acts as an absolute reference. It measures the strength and direction of the Earth's magnetic field, allowing the system to establish a global "North"

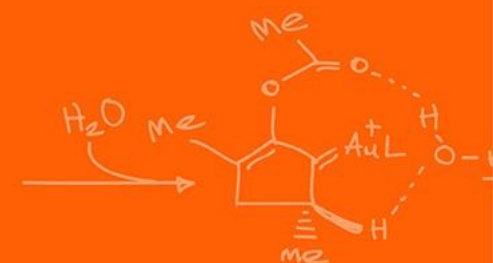
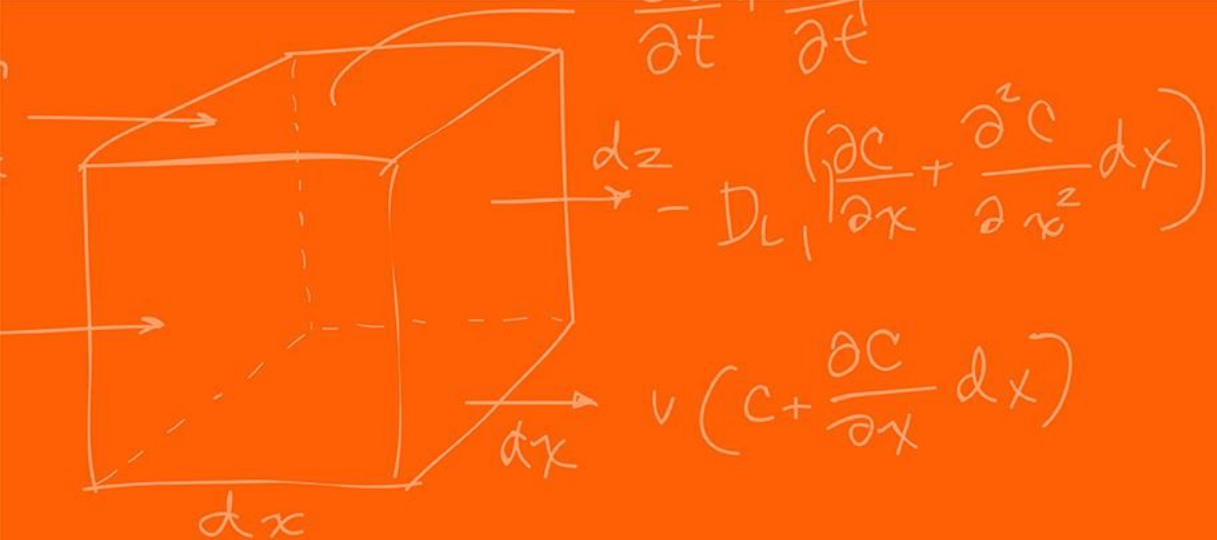
The more popular method uses the Hall Effect sensor to measure the magnetic field; however, next slide is a slightly different, method, and was chosen since it is more like the previous accelerometer and gyroscope sensors. Although they measure the same thing in the end!

The magnetic field acts like a counterbalance and you measure the torque

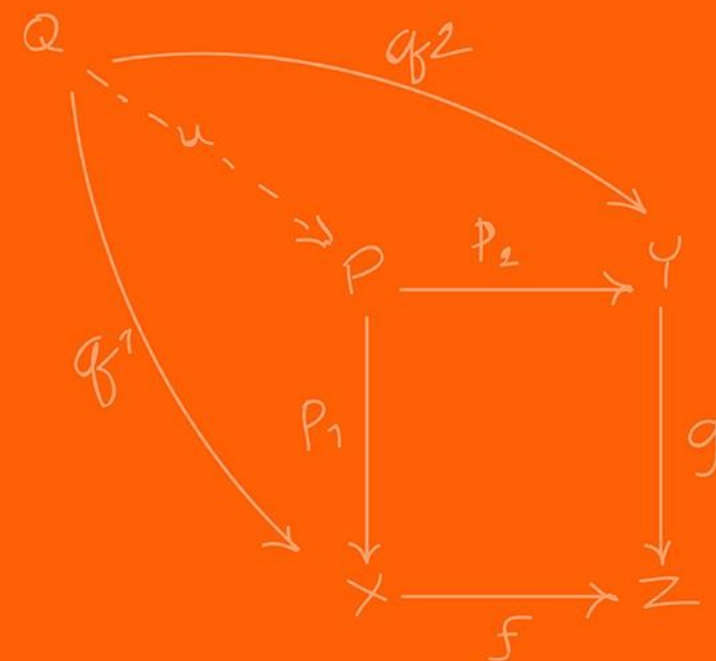


https://www.researchgate.net/figure/Illustration-of-a-ferromagnetic-MEMS-magnetometer-design_fig18_255663586





IMU



IMU – stands for Inertial Measurement Unit

Which is just a single chip that packages 3-axis accelerometers, 3-axis gyroscopes and a 3-axis magnetometer (sometimes).

Sometimes includes additional noise filtering, sensor fusion.

Allows to collect all the information about the robot's inertial state.

We will specially be using the MPU-9250. This specifically contains a 3-axis accelerometer, 3-axis gyroscope and a 3-axis magnetometer.

There are multiple sensitivity levels that you can set the sensors:

- Gyroscope: ± 250 , ± 500 , ± 1000 , and ± 2000 dps (degrees per second)
- Accelerometer: $\pm 2g$, $\pm 4g$, $\pm 8g$, and $\pm 16g$
- Magnetometer: $\pm 4800\mu T$ (Tesla)

We can communicate to it through I2C at 400kHz or SPI at 1MHz (up to 20MHz).

These sensors are meant to be read extremely fast, why is that the case?

We need to integrate to get velocity and position so the smaller the dt the more accurate!

We will be communicating through SPI.

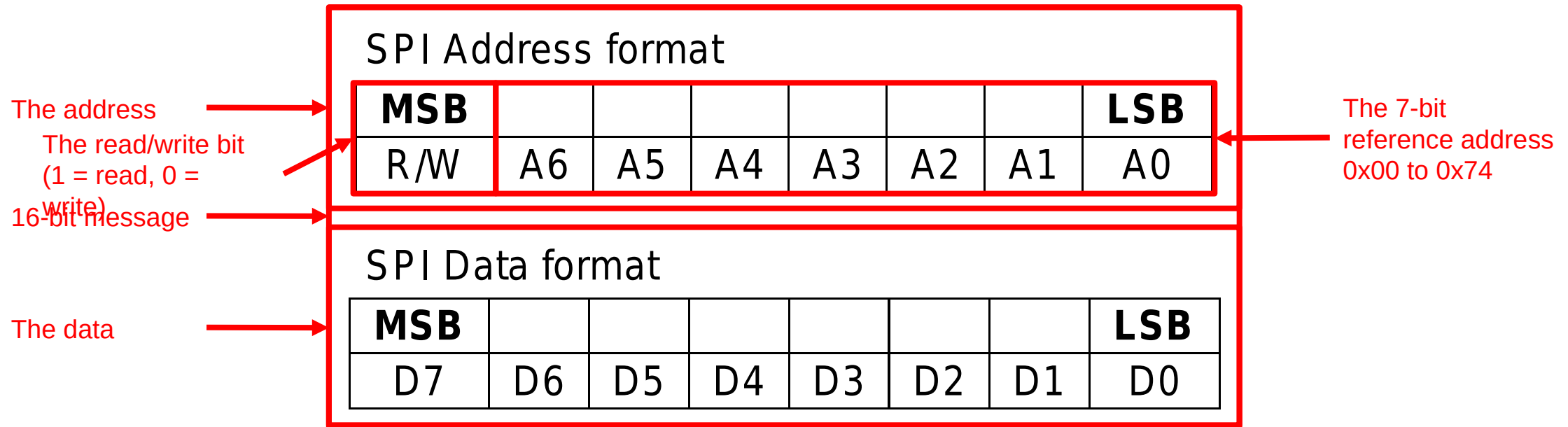
Each the returned results will be 16-bit signed integers. As such:

Acceleration X, Y, Z: $[-32768, 32767] = [-4g, 4g]$

Gyro X, Y, Z: $[-32768, 32767] = [-250 \text{ dps}, 250 \text{ dsp}]$

There are 2 components for this sensor, writing configurations vs reading data. This is encoded inside the SPI data. We separate the message into 2 8-bit parts.

The **address (top 8-bits)**: Says if we are trying to read/write and from where (accel vs gyro)
The **data (bottom 8-bits)**: what we are trying to send to write to that address (ignored if reading)



Addr (Hex)	Addr (Dec.)	Register Name	Serial I/F	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
35	53	I2C_SLV4_DI	R	I2C_SLV4_DI[7:0]							
36	54	I2C_MST_STATUS	R	PASS_THROUGH	I2C_SLV4_DONE	I2C_LOST_ARB	I2C_SLV4_NACK	I2C_SLV3_NACK	I2C_SLV2_NACK	I2C_SLV1_NACK	I2C_SLV0_NACK
37	55	INT_PIN_CFG	R/W	ACTL	OPEN	LATCH_INT_EN	INT_ANYR_D_2CLEAR	ACTL_FSYNC	FSYNC_INT_MODE_EN	BYPASS_EN	-
38	56	INT_ENABLE	R/W	-	WOM_EN	-	FIFO_OFLOW_EN	FSYNC_INT_EN	-	-	RAW_RDY_EN
3A	58	INT_STATUS	R	-	WOM_INT	-	FIFO_OFLOW_INT	FSYNC_INT	-	-	RAW_DATA_RDY_INT
3B	59	ACCEL_XOUT_H	R	ACCEL_XOUT_H[15:8]							
3C	60	ACCEL_XOUT_L	R	ACCEL_XOUT_L[7:0]							
3D	61	ACCEL_YOUT_H	R	ACCEL_YOUT_H[15:8]							
3E	62	ACCEL_YOUT_L	R	ACCEL_YOUT_L[7:0]							
3F	63	ACCEL_ZOUT_H	R	ACCEL_ZOUT_H[15:8]							
40	64	ACCEL_ZOUT_L	R	ACCEL_ZOUT_L[7:0]							
41	65	TEMP_OUT_H	R	TEMP_OUT_H[15:8]							
42	66	TEMP_OUT_L	R	TEMP_OUT_L[7:0]							
43	67	GYRO_XOUT_H	R	GYRO_XOUT_H[15:8]							
44	68	GYRO_XOUT_L	R	GYRO_XOUT_L[7:0]							
45	69	GYRO_YOUT_H	R	GYRO_YOUT_H[15:8]							
46	70	GYRO_YOUT_L	R	GYRO_YOUT_L[7:0]							
47	71	GYRO_ZOUT_H	R	GYRO_ZOUT_H[15:8]							
48	72	GYRO_ZOUT_L	R	GYRO_ZOUT_L[7:0]							
49	73	EXT_SENS_DATA_00	R	EXT_SENS_DATA_00[7:0]							
4A	74	EXT_SENS_DATA_01	R	EXT_SENS_DATA_01[7:0]							
4B	75	EXT_SENS_DATA_02	R	EXT_SENS_DATA_02[7:0]							
4C	76	EXT_SENS_DATA_03	R	EXT_SENS_DATA_03[7:0]							
4D	77	EXT_SENS_DATA_04	R	EXT_SENS_DATA_04[7:0]							

The address →

If you can read / write to this address

Data is split into 8-bit high and low bits

As you previously saw we want to write 16-bits of data and potentially read 16-bits at a time to reduce the complexity of having to bit shift the high and low bits into the same variable.

To do this what would we need to set SPICHR to?

0xA 0x11 0xF 0x8

When writing configuration to the MPU board, such as the accelerometer X offset (initial bias)

```
GpioDataRegs.GPCCLEAR.bit.GPIO66 = 1;    // Start SPI
// 0x77 = 0111 0111 is this read or writing?
SpibRegs.SPITXBUF = (0x7700 | 0x00EB);    // Address + data
while(SpibRegs.SPIFFRX.bit.RXFFST !=1);  // Waiting for
response
GpioDataRegs.GPCSET.bit.GPIO66 = 1;      // Disable SPI
temp = SpibRegs.SPIRXBUF;                  // Read value even
though not needed
DELAY_US(10);    // Delay a bit before issuing another MPU-9250
SPI transfer
```


What if we want to write multiple pieces of data at once?

```
GpioDataRegs.GPCCLEAR.bit.GPIO66 = 1;
SpibRegs.SPITXBUF = (0x1900 | 0x0013); // One write
SpibRegs.SPITXBUF = (0x0200 | 0x0000); // Second write
while(SpibRegs.SPIFFRX.bit.RXFFST !=2); // Wait that both have been sent
GpioDataRegs.GPCSET.bit.GPIO66 = 1;
temp = SpibRegs.SPIRXBUF; // Read value even though not needed
temp = SpibRegs.SPIRXBUF; // Read second value even though not needed
DELAY_US(10); // Delay a bit before issuing another MPU-9250 SPI transfer
```

What is the maximum number of writes at a time?

The maximum size of the TX/RX FIFO is 16!

When explicitly wanting to read the data, especially if there is a lot we want to read, accel X, Y, Z, gyro X, Y, Z, each of them having a high and low address. Is there a smart way of doing this?

Yes!

This MPU you send an address that you want to **start** reading from. If you continue to give it clock cycles it will just **continue down to the next address** and read down!

Allows us to read the 16-bit data into a single 16-bit read!

Addr (Hex)	Addr (Dec.)	Register Name	Serial I/F	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
35	53	I2C_SLV4_DI	R	I2C_SLV4_DI[7:0]							
36	54	I2C_MST_STATUS	R	PASS_THROUGH	I2C_SLV4_DONE	I2C_LOST_ARB	I2C_SLV4_NACK	I2C_SLV3_NACK	I2C_SLV2_NACK	I2C_SLV1_NACK	I2C_SLV0_NACK
37	55	INT_PIN_CFG	R/W	ACTL	OPEN	LATCH_INT_EN	INT_ANYR_D_2CLEAR	ACTL_FSYNC	FSYNC_INT_MODE_EN	BYPASS_EN	-
38	56	INT_ENABLE	R/W	-	WOM_EN	-	FIFO_OFLOW_EN	FSYNC_INT_EN	-	-	RAW_RDY_EN
3A	58	INT_STATUS	R	-	WOM_INT	-	FIFO_OFLOW_INT	FSYNC_INT	-	-	RAW_DATA_RDY_INT
3B	59	ACCEL_XOUT_H	R	ACCEL_XOUT_H[15:8]							
3C	60	ACCEL_XOUT_L	R	ACCEL_XOUT_L[7:0]							
3D	61	ACCEL_YOUT_H	R	ACCEL_YOUT_H[15:8]							
3E	62	ACCEL_YOUT_L	R	ACCEL_YOUT_L[7:0]							
3F	63	ACCEL_ZOUT_H	R	ACCEL_ZOUT_H[15:8]							
40	64	ACCEL_ZOUT_L	R	ACCEL_ZOUT_L[7:0]							
41	65	TEMP_OUT_H	R	TEMP_OUT_H[15:8]							
42	66	TEMP_OUT_L	R	TEMP_OUT_L[7:0]							
43	67	GYRO_XOUT_H	R	GYRO_XOUT_H[15:8]							
44	68	GYRO_XOUT_L	R	GYRO_XOUT_L[7:0]							
45	69	GYRO_YOUT_H	R	GYRO_YOUT_H[15:8]							
46	70	GYRO_YOUT_L	R	GYRO_YOUT_L[7:0]							
47	71	GYRO_ZOUT_H	R	GYRO_ZOUT_H[15:8]							
48	72	GYRO_ZOUT_L	R	GYRO_ZOUT_L[7:0]							
49	73	EXT_SENS_DATA_00	R	EXT_SENS_DATA_00[7:0]							
4A	74	EXT_SENS_DATA_01	R	EXT_SENS_DATA_01[7:0]							
4B	75	EXT_SENS_DATA_02	R	EXT_SENS_DATA_02[7:0]							
4C	76	EXT_SENS_DATA_03	R	EXT_SENS_DATA_03[7:0]							
4D	77	EXT_SENS_DATA_04	R	EXT_SENS_DATA_04[7:0]							

Start reading
(sent 8-bit start
address)

Continue reading
16-bits
Continue reading
16-bits
Continue reading
16-bits

```
// Code inside CPU Timer 0
GpioDataRegs.GPCCLEAR.bit.GPI066 = 1;
SpibRegs.SPIFFRX.bit.RXFFIL = 3;
// 0x8 = 1000 = read
// 0x3C = address to read from
SpibRegs.SPITXBUF = ((0x8000) |
(0x3C00));
SpibRegs.SPITXBUF = 0; // 0x0000
continue
SpibRegs.SPITXBUF = 0; // 0x0000
continue
```

```
int16_t dummy= 0;
int16_t accelYraw = 0;
int16_t accelZraw = 0;
float accelYreading = 0;
float accelZreading = 0;
__interrupt void SPIB_isr(void){
    GpioDataRegs.GPCSET.bit.GPI066 = 1; // Stop SPI transmit
    dummy = SpibRegs.SPIRXBUF; // reads from 0x3C Accel x low only
    accelYraw = SpibRegs.SPIRXBUF; // reads 0x3D | 0x3E = Accel y
out
    accelZraw = SpibRegs.SPIRXBUF; // reads 0x3F | 0x40 = Accel z
out
    accelYreading = accelYraw*4.0/32767.0;
    accelZreading = accelZraw*4.0/32767.0;
    SpibRegs.SPIFFRX.bit.RXFFOVFCLR=1; // Clear Overflow flag
    SpibRegs.SPIFFRX.bit.RXFFINTCLR=1; // Clear Interrupt flag
    PieCtrlRegs.PIEACK.all = PIEACK_GROUP6;
}
```

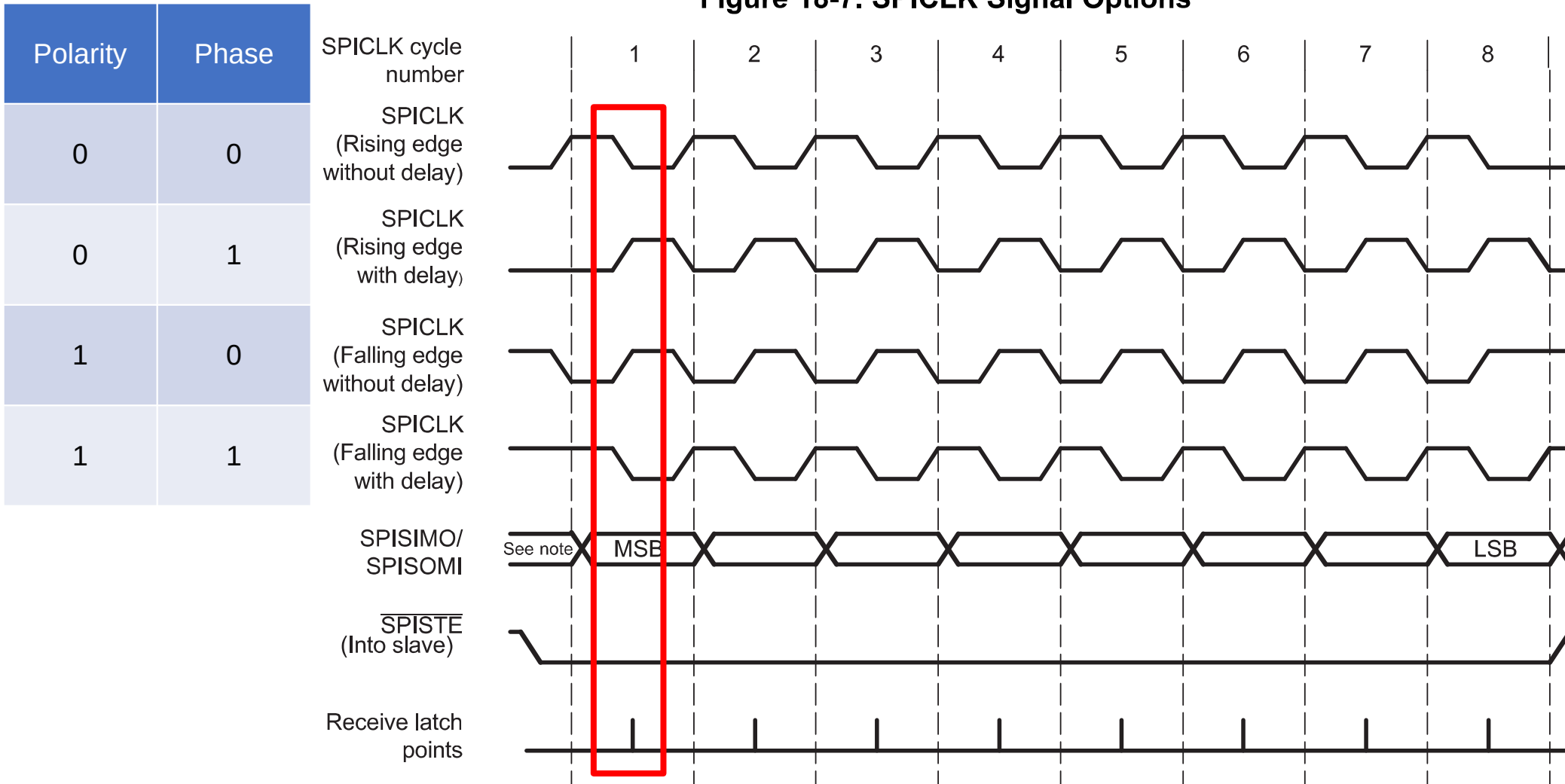
Addr (Hex)	Addr (Dec.)	Register Name	Serial I/F	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
35	53	I2C_SLV4_DI	R	I2C_SLV4_DI[7:0]							
36	54	I2C_MST_STATUS	R	PASS_THROUGH	I2C_SLV4_DONE	I2C_LOST_ARB	I2C_SLV4_NACK	I2C_SLV3_NACK	I2C_SLV2_NACK	I2C_SLV1_NACK	I2C_SLV0_NACK
37	55	INT_PIN_CFG	R/W	ACTL	OPEN	LATCH_INT_EN	INT_ANYRD_2CLEAR	ACTL_FSYNC	FSYNC_INT_MODE_EN	BYPASS_EN	-
38	56	INT_ENABLE	R/W	-	WOM_EN	-	FIFO_OFLOW_EN	FSYNC_INT_EN	-	-	RAW_RDY_EN
3A	58	INT_STATUS	R	-	WOM_INT	-	FIFO_OFLOW_INT	FSYNC_INT	-	-	RAW_DATA_RDY_INT
3B	59	ACCEL_XOUT_H	R	ACCEL_XOUT_H[15:8]							
3C	60	ACCEL_XOUT_L	R	ACCEL_XOUT_L[7:0]							
3D	61	ACCEL_YOUT_H	R	ACCEL_YOUT_H[15:8]							
3E	62	ACCEL_YOUT_L	R	ACCEL_YOUT_L[7:0]							
3F	63	ACCEL_ZOUT_H	R	ACCEL_ZOUT_H[15:8]							
40	64	ACCEL_ZOUT_L	R	ACCEL_ZOUT_L[7:0]							
41	65	TEMP_OUT_H	R	TEMP_OUT_H[15:8]							
42	66	TEMP_OUT_L	R	TEMP_OUT_L[7:0]							
43	67	GYRO_XOUT_H	R	GYRO_XOUT_H[15:8]							
44	68	GYRO_XOUT_L	R	GYRO_XOUT_L[7:0]							
45	69	GYRO_YOUT_H	R	GYRO_YOUT_H[15:8]							
46	70	GYRO_YOUT_L	R	GYRO_YOUT_L[7:0]							
47	71	GYRO_ZOUT_H	R	GYRO_ZOUT_H[15:8]							
48	72	GYRO_ZOUT_L	R	GYRO_ZOUT_L[7:0]							
49	73	EXT_SENS_DATA_00	R	EXT_SENS_DATA_00[7:0]							
4A	74	EXT_SENS_DATA_01	R	EXT_SENS_DATA_01[7:0]							
4B	75	EXT_SENS_DATA_02	R	EXT_SENS_DATA_02[7:0]							
4C	76	EXT_SENS_DATA_03	R	EXT_SENS_DATA_03[7:0]							
4D	77	EXT_SENS_DATA_04	R	EXT_SENS_DATA_04[7:0]							

```

SpibRegs.SPITXBUF = ((0x8000) |
(0x3C00));
SpibRegs.SPITXBUF = 0; // 0x0000
continue
SpibRegs.SPITXBUF = 0; // 0x0000
continue

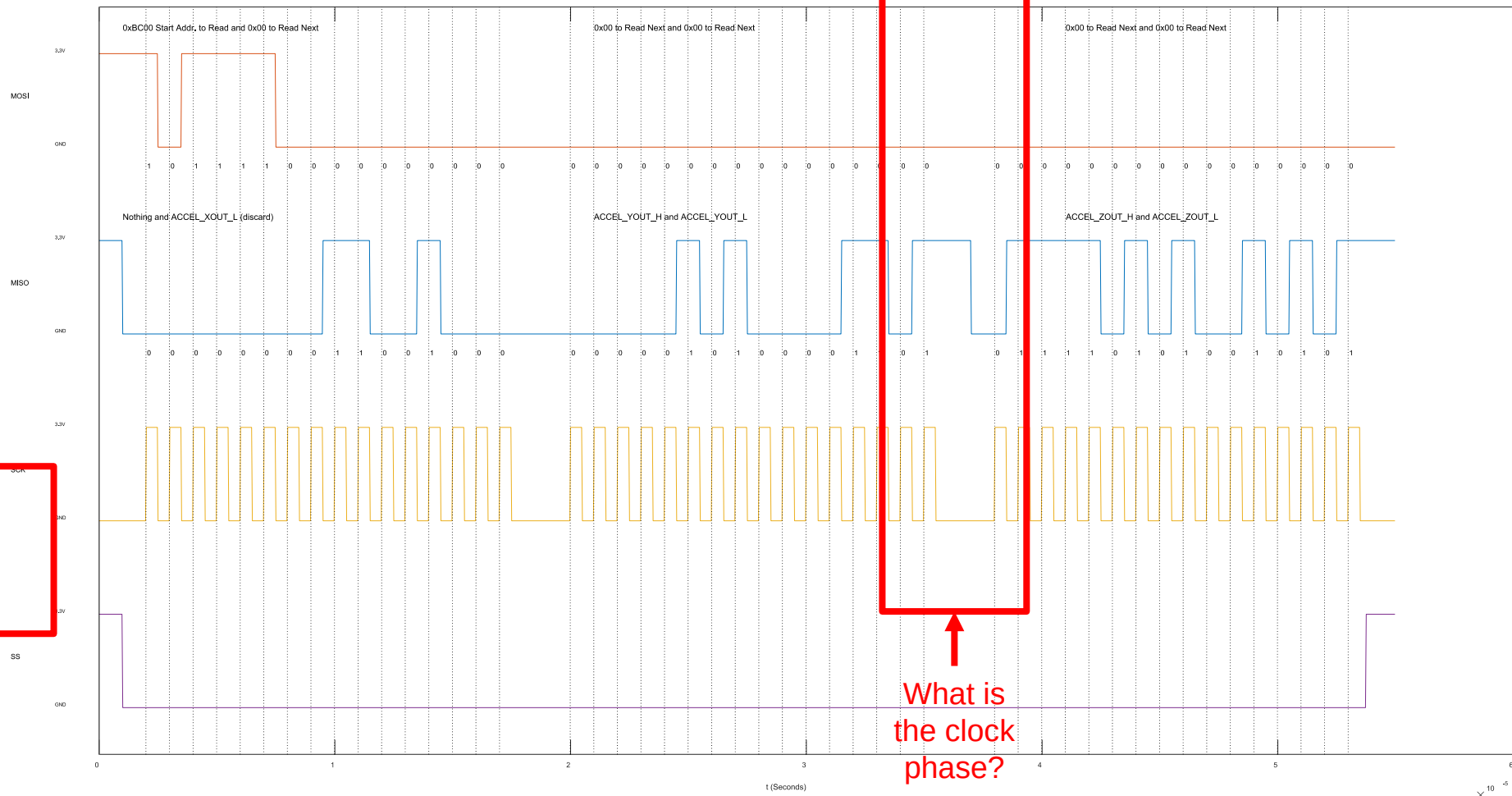
```

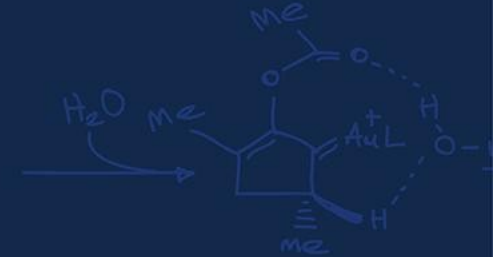
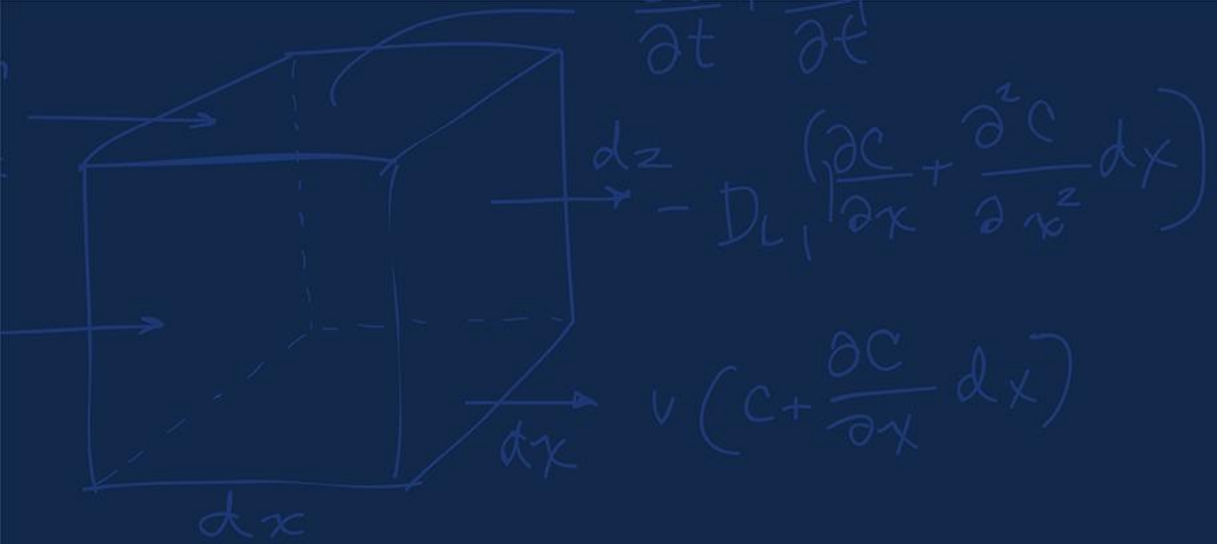
Figure 18-7. SPICLK Signal Options



Note: Previous data bit

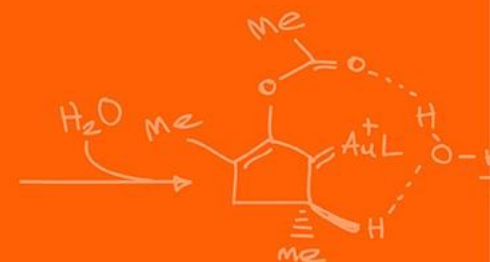
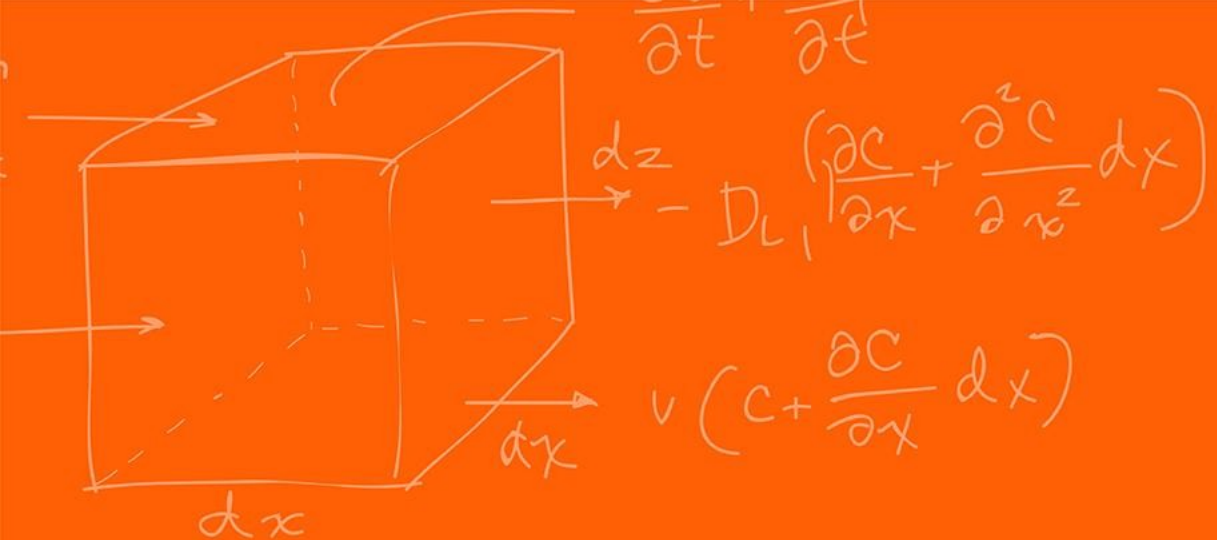
MPU9250 SPI Timing Diagram Read AccelYH AccelYL AccelZH AccelZL as two 16bit values (MSB First) CLK POLARITY=0 and PHASE=1





Questions?





The Grainger College of Engineering

UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

